EPITA · DevOps capstone · Team report

FulfillOps: architecture, delivery and operational evidence

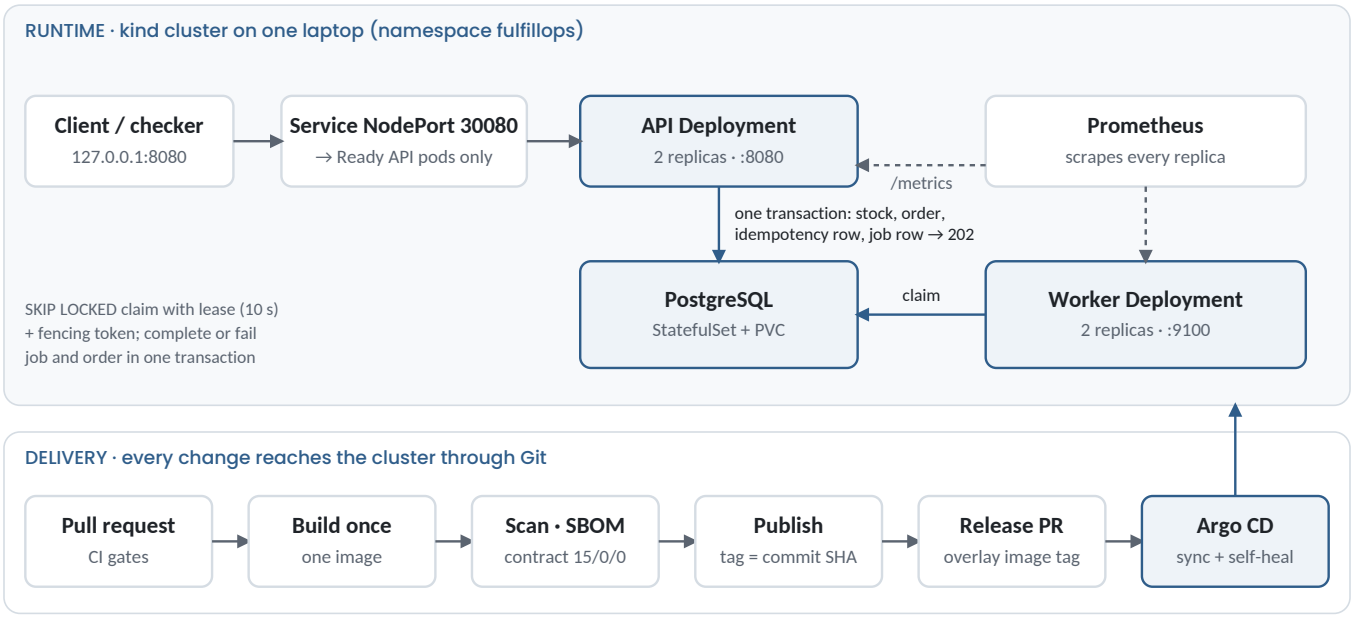
FulfillOps — resilient order processing · 14 November 2026 · Person A (API & persistence) · Person B (worker & reliability) · Person C (delivery & operations)

> **Evidence boundary:** "Observed" below means a committed evidence artifact or a completed CI run. "Design intent" means a contract, code path, or manifest that describes expected behavior; it is not by itself proof that a failure scenario was exercised. Repository references are abbreviated inline. The latest cited main-branch CI run is run 43508189254, for commit `dc6fee8ed4e79392c4202270e43c1d37da10475a`.

15 / 0 / 0	58	8 → 1	120 / 120	599 / 599	2 → 2
public checker in CI pass / fail / skip	**tests passed in CI** fresh PostgreSQL	**same-key requests** → 1 order · 2 API replicas	**probes OK** good rolling update	**probes OK** broken release + revert	**orders before / after** no committed-order loss

1. Architecture

System view



FulfillOps accepts an order through a stateless API, stores the business state and work request in PostgreSQL, and completes the work asynchronously in independent worker processes. PostgreSQL is both the durable application store and the queue. Prometheus scrapes every API and worker replica separately. The same image provides API, worker, and migration roles selected by the process argument; replicas scale the API and worker independently. In Kubernetes the API and worker Deployments each request two replicas; Postgres is a StatefulSet with a persistent volume claim.

Source · **design intent**: `docs/architecture.md`, `docs/integration-contract.md`, `infra/k8s/base/`

The client-facing local path is `127.0.0.1:8080` → kind host mapping → node port 30080 → API Service → ready API pod. In Compose, nginx owns host port 8080 and balances across API replicas. In kind, there is no nginx; the Kubernetes Service routes the NodePort. Prometheus reaches each replica through headless metrics Services and DNS service discovery. The kind mapping binds only to loopback so that the development service is reachable from the laptop but not exposed to its LAN.

Source · **design intent**: `infra/kind/cluster.yaml`, `infra/k8s/base/api-service.yaml`, `infra/k8s/base/api-metrics-service.yaml`, `infra/k8s/base/worker-metrics-service.yaml`, `compose.yaml`, `infra/compose/nginx.conf`

Transaction and invariants

Order acceptance is the key consistency boundary. The API validates and canonicalizes the payload, locks inventory rows in a stable order, checks available quantities, decreases stock, and inserts the order, order lines, idempotency mapping, and one job row in the same database transaction. The API returns `202` only after commit. A client retry with the same key and equivalent request receives the original order identifier and does not reserve stock again. Reusing a key with a different payload conflicts. Competing requests with different keys serialize on inventory rows, so committed reservations cannot exceed stock.

Source · **design intent**: `docs/integration-contract.md` §§2–4, `api/main.py`, `api/orders.py`, `api/idempotency.py`, `api/models.py`

PostgreSQL mechanisms enforce these rules rather than process memory. The idempotency key is a primary key; each job has a unique order identifier; row locks protect stock checks and updates; foreign keys connect order details and jobs. The transaction ensures a failure cannot persist only a reservation or only an order. The catalog contains KIT-100 at 1299 cents, PART-200 at 599 cents, and PACK-300 at 249 cents. Fresh-database seed quantities are 7, 12, and 5 respectively.

Source · **design intent**: `docs/API_CONTRACT_EN.md`, `db/migrations/`, `api/models.py`

The API contract makes the reservation point visible to clients: stock decreases when an order is accepted, before asynchronous completion. A queued response is therefore a durable receipt, not a promise that the worker has already finished. GET order status separates `queued`, `processing`, `completed`, and `failed`; a failed job remains inspectable rather than being deleted. Invalid quantities, unknown SKUs, and insufficient stock have distinct status and error-code responses, allowing clients and the checker to tell caller errors from contention. The public contract also caps request body size and validates idempotency keys, reducing accidental ambiguity at the transaction boundary.

Source · **design intent**: `docs/API_CONTRACT_EN.md`, `api/`

Workers claim eligible jobs with `FOR UPDATE SKIP LOCKED`, record a worker identity, lease expiry, and fencing token, then commit the claim before processing. A lease permits recovery after a process disappears; the token prevents a former worker from committing after its lease was replaced. The agreed policy is a 10-second lease renewed about every 3 seconds, at most three attempts including the first, and exponential retry delay capped at 8 seconds. Completion updates the job and order in a database transaction. A permanently failing job remains failed and auditable; stock is not silently restored.

Source · **design intent**: `docs/integration-contract.md` §§5–7, `worker/`, `db/migrations/`

PostgreSQL was selected over a separate broker to keep durable order state, stock reservation, idempotency, and work creation in one transactional system. `SKIP LOCKED` lets multiple workers claim different jobs without a separate queue service. This reduces infrastructure but couples queue throughput and application data to PostgreSQL capacity. The API and worker share one versioned image with role arguments, reducing divergence between the deployed roles. Two replicas of each are the target topology: API replicas serve requests, and worker replicas provide concurrent claims and a peer for recovery.

Source · **design intent**: `docs/ADR-001.md`, `docs/integration-contract.md`, `infra/k8s/base/api-deployment.yaml`, `infra/k8s/base/worker-deployment.yaml`

Liveness is process-only; readiness checks database access and migration state. They deliberately answer different questions: a database outage should remove an API from ready traffic without causing the orchestrator to restart a live process. Worker health is served on port 9100 and does not expose database credentials. API and worker metrics are process-local measurements; Prometheus performs cross-replica aggregation. Logs are JSON to stdout, and identifiers or credentials are not metric labels.

Source · **design intent**: `docs/integration-contract.md`, `api/main.py`, `api/metrics.py`, `worker/main.py`, `infra/k8s/base/`

2. Tests and contract evidence

The test approach has four layers. Unit and repository tests exercise validation, state transitions, database access, retry boundaries, and worker claim logic. API tests exercise public HTTP statuses and schemas, including liveness/readiness, catalog and inventory, idempotency replay, conflicts, invalid requests, insufficient stock, and metrics. Integration tests use PostgreSQL and migrations to check behavior across the real persistence boundary. Deployment checks run the built image and public checker against an isolated Compose project. CI's PostgreSQL service is a fresh `fulfillops_test` database; it applies Alembic migrations before the full suite.

Source · **design intent**: `tests/`, `.github/workflows/ci.yml`; OBSERVED: CI run 43508189254

Observed in CI: run 43508189254 collected 58 tests and finished `58 passed, 1 warning in 33.00s`. The same run's contract job started the isolated test stack, checked readiness and catalog, verified the checker at `scripts/fulfillops_public_checker.py`, ran it with `--concurrency --stress`, and recorded `RESULT: 15 passed / 0 failed / 0 skipped`. The stack was then torn down with its disposable test volume. This proves that the checked-in real application passed the checker on that run; it does not prove the same result for every future source revision.

Source · CI run 43508189254, test and contract jobs; `.github/workflows/ci.yml`

The API race tests use eight concurrent calls for an identical idempotency key and eight distinct keys competing for the seven seeded KIT-100 units. The identical-key test expects one order identifier and one unit reserved; the distinct-key test expects seven accepted calls, one `OUT_OF_STOCK`, and no available stock left. They are independent test cases, so together they distinguish safe retry from oversell prevention.

Source · **design intent** / test assertions: `tests/api/test_api.py::test_concurrent_idempotent_requests`, `tests/api/test_api.py::test_concurrent_orders_do_not_oversell`

Observed Gate 1 proof: `docs/evidence/gate1-idempotency-20261113.txt` records eight simultaneous identical-key requests through two healthy API replicas; all returned the same order ID, and database checks showed one order, one job, and one idempotency record. It then records competing distinct-key requests against limited inventory and a replay after an API replica restart returning the original order. The artifact says "GATE 1 RESULT: ALL CHECKS PASSED." This is stronger than the unit test because the requests pass through a running multi-replica service and the proof inspects persisted database state.

Source · **observed**: `docs/evidence/gate1-idempotency-20261113.txt`, PR #30

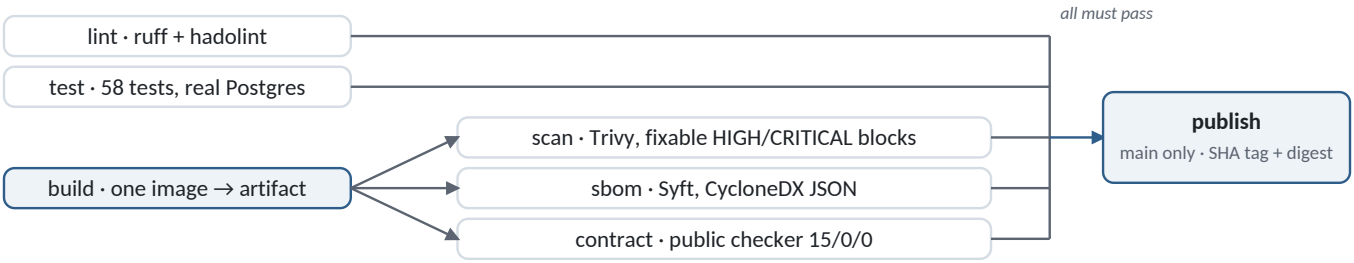
Observed worker reliability evidence: the committed local bundle reports four passing scenarios: two real worker processes with a killed peer and recovery, a poison-processing callback reaching durable failure, a third crashed attempt becoming failed without a fourth claim, and SIGTERM during active completion finishing without another claim. Its pytest output is `4 passed in 7.24s`. Scope matters: the bundle explicitly says these were local Python processes with real PostgreSQL, not a container or Kubernetes drill. Some expiry/backoff transitions were advanced by fixture SQL. It therefore supports the worker state-machine behavior but is not evidence of a live cluster worker-kill exercise.

Source · **observed**: `docs/evidence/worker-20261113T092847Z-7e0a2f/summary.json`, `test-output.txt`

The supply-chain red/green exercise is separate from the application test-development criterion. PR #14 added a deliberately vulnerable `urllib3==1.26.4`, for which Trivy reported HIGH CVE-2021-33503 with fixed version 1.26.5; the blocking scan failed and publish did not proceed. A follow-up commit removed the package, after which the scan passed. This proves that the configured image vulnerability gate blocks a known fixable vulnerability. It does **not** show an application test failing on a business bug and then passing after a code fix.

Source · **observed**: `docs/evidence/red-green.md`, PR #14, runs 43080402689 and 43080798851

3. CI pipeline and supply chain



The CI workflow runs on pull requests to `main`, pushes to `main`, and manual dispatch. The graph is: lint and tests in parallel with one image build; then Trivy scan, CycloneDX SBOM generation, and isolated public contract checks consume the same built-image artifact; publish waits for all five quality jobs and runs only for a push to `main`. This avoids building a different image for each gate.

Source · **design intent**: `.github/workflows/ci.yml`, `docs/ci-cd.md`

Ruff checks and formats the application, tests, and scripts; Hadolint checks Dockerfiles. The test job installs the hash-locked requirements, applies migrations, and runs `pytest tests/` against PostgreSQL 16.10. The contract job constructs a masked disposable environment, launches Compose without rebuilding, waits for readiness, checks liveness/readiness/catalog, and runs the public checker including concurrency and stress. Cleanup runs even when the checker fails.

Source · **design intent**: `.github/workflows/ci.yml`

The blocking Trivy invocation considers OS and library vulnerabilities with severity HIGH or CRITICAL, ignores findings without a fix, and returns nonzero when a fixable finding is present. A second all-severity report is informational. The red/green run in PR #14 is the observed demonstration of the blocking policy. Syft's action produces a CycloneDX JSON SBOM from the built image; CI uploads it and attaches it to the published-image workflow artifacts. For commit `dc6fee8`…, run 43508189254 completed the build, scan, SBOM, contract, and publish jobs successfully.

Source · **design intent**: `.github/workflows/ci.yml`, `docs/ci-cd.md`; OBSERVED: `docs/evidence/red-green.md`, CI run 43508189254

Workflow actions are pinned to full commit SHAs with version comments. Workflow-level permissions default to `contents: read`; only publish receives `packages: write`. The publish job tags the already-tested image with the full Git commit SHA and records the registry manifest digest. For `dc6fee8ed4e79392c4202270e43c1d37da10475a`, the main publish log in run 43508189254 reports `sha256:17efd779466c5d3c238ade910506297c1306733b0334d9d0a73f19c3ff28ab36`.

Source · `.github/workflows/ci.yml`; OBSERVED: CI run 43508189254 publish job

Branch protection on `main` was configured by the repository administrator: pull requests are required, the `lint`, `test`, `build`, `scan`, `sbom` and `contract` checks must pass, and the rule cannot be bypassed. GitHub exposes protection settings only to admins, so the authors' write-level token could not re-read them through the API at report time (it returns 404); independently of the settings, the workflow makes publish depend on every quality job.

Source · **design intent**: `docs/ci-cd.md`, `.github/workflows/ci.yml`; repository settings (admin)

4. Deployment and operations

The local Kubernetes environment is kind: Kubernetes nodes run as Docker containers on one Mac. The cluster definition uses one control-plane node and one worker node, a pinned kindest/node image, and an extra port mapping from host loopback port 8080 to node port 30080. The node port is served by a Kubernetes Service, which selects ready API pods. The control-plane has a `NoSchedule` taint; application pods are intended to run on the worker.

Source · **design intent**: `infra/kind/cluster.yaml`, `docs/kubernetes-local.md`, `docs/environment.md`

Kustomize has a shared base and a kind overlay. The base defines API and worker Deployments, Services, Postgres StatefulSet and PVC, the migration Job, configuration, and Prometheus. The overlay chooses the kind image tag. API and worker run as UID 10001, with read-only root filesystems, dropped capabilities, no privilege escalation, and writable temporary directories only where required. API uses startup, readiness, and liveness probes; its rolling strategy permits one surge and zero unavailable replicas. Worker probes use port 9100, and worker pods receive an eight-second termination grace period.

Source · **design intent**: `infra/k8s/base/`, `infra/k8s/overlays/kind/`, `docs/kubernetes-manifests.md`

Configuration is separated by sensitivity. Non-secret runtime values live in a ConfigMap; database credentials and the connection string are supplied through a Secret generated locally by the bootstrap procedure. Pods do not receive a Kubernetes API token unless needed. Application roles are selected with image arguments so the image entrypoint can dispatch the role and use `exec` for signal delivery. These choices reduce accidental credential exposure and preserve the process lifecycle behavior expected by Kubernetes.

Source · **design intent**: `infra/k8s/base/`, `scripts/bootstrap-secrets.sh`, `Dockerfile`, `docs/integration-contract.md`

PostgreSQL uses a StatefulSet and persistent claim; pruning is disabled for the claim so routine GitOps reconciliation does not delete database storage. The migration Job is an Argo CD Sync hook at wave 1, after Postgres wave 0 and before API/worker wave 2. `BeforeHookCreation` replaces the prior hook Job on sync; failed Job pods are retained for diagnosis and finished Jobs expire after one hour. The migration command is `alembic upgrade head`. Seeding is migration-time and runs once through Alembic versioning. The contract documents no advisory lock because the deployment path runs one migration Job at a time.

Source · **design intent**: `infra/k8s/base/migrate-job.yaml`, `infra/k8s/base/postgres-statefulset.yaml`, `docs/gitops.md`, `docs/integration-contract.md`

Argo CD watches the repository's main branch and the kind overlay. AppProject limits the source and destination; automated sync uses self-heal, while pruning is restricted to protect durable database data. Application changes go through Git, a reviewed PR, and Argo reconciliation; the release helper builds the exact requested commit and opens a Git change to the overlay. This gives each deployed image a traceable source SHA and avoids leaving a manual cluster patch as the lasting state.

Source · **design intent**: `infra/argocd/`, `scripts/release.sh`, `docs/gitops.md`

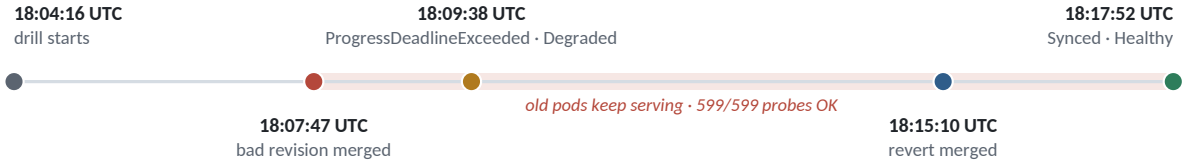
Observed drift recovery: a committed drift artifact records the API replica count being changed manually from two to one and Argo restoring two replicas, with the application returning to Synced/Healthy. **Observed rolling update:** the complete run recorded 120 probe samples across 30 seconds, zero failures, and copied the CSV to `docs/rolling_update_probe.csv`.

Source · **observed**: `docs/evidence/drift-20261113T174208Z.txt`, `docs/evidence/rolling-update-20261113T180034Z/summary.md`, `docs/rolling_update_probe.csv`

Observed metrics: Prometheus evidence records four up targets: two API and two worker replicas. The script sent 250 catalog GETs, then calculated request rate using `sum(rate(fulfillops_http_requests_total[1m]))` and p95 using `histogram_quantile(0.95, sum by (le) (rate(fulfillops_http_request_duration_seconds_bucket[1m])))`. The stored p95 is about 0.00813 seconds. The artifact explicitly uses GET requests because order POSTs consume finite inventory. These are local kind measurements over a short exercise, not a production latency SLO.

Source · **observed**: `docs/evidence/prometheus-20261113T174220Z.txt`

5. Reliability incident: controlled broken release



This was a planned failure-injection drill, not an unplanned customer incident. The drill introduced an API revision whose readiness probe used `/health/broken`. With `maxUnavailable: 0`, the existing ready replicas remained in service while the new pod failed readiness. Kubernetes stopped the rollout after its progress deadline, and Argo CD reported the application Degraded. The point of the exercise was to test detection and recovery through Git, not to measure the frequency of an actual outage.

Source · **observed**: `docs/evidence/broken-release-20261113T180416Z/summary.md`, `docs/incident-postmortem.md`

The evidence gives this timeline in UTC: the drill started at 18:04:16 on 13 November 2026; the bad revision merged at 18:07:47; progression halted at 18:09:38 after 90 seconds with `ProgressDeadlineExceeded` and Argo Degraded; the revert merged at 18:15:10; and Argo was Healthy again at 18:17:52. The known-good revision was `4c858636`…; the bad revision was `779751ba`…; recovery landed at `1bf6538`….

Source · **observed**: `docs/evidence/broken-release-20261113T180416Z/summary.md`

The operational impact recorded by the drill was no failed availability probe: 599 samples, zero failures. The order count was two before and two after. The evidence compares the recovered manifests to the known-good revision and records an empty diff. These results describe the controlled drill and its probe path; they do not establish availability for all clients or all failure modes.

Source · **observed**: `docs/evidence/broken-release-20261113T180416Z/summary.md`, `known-good-manifest-diff.txt`

Recovery used a Git revert rather than a fix-forward because the previous revision was known good, the cause of the introduced readiness defect was already isolated, and no schema migration had occurred. The revert flowed through a PR and Argo sync, preserving Git as the source of truth. The immediate prevention measure is the rollout configuration itself: readiness must succeed before a replacement pod is ready, and `maxUnavailable: 0` preserves old replicas while the replacement is unhealthy. A further operational lesson is to retain migration failure pods and their logs; the migration Job now uses `restartPolicy: Never`, keeps its retry limit, and has a one-hour TTL.

Source · **observed / design intent**: `docs/incident-postmortem.md`, `infra/k8s/base/api-deployment.yaml`, `infra/k8s/base/migrate-job.yaml`, PRs #28–#29

The worker reliability bundle is related but is not evidence from this deployment incident. It exercises killed-worker recovery, poison-job terminal failure, and graceful SIGTERM in local processes against PostgreSQL, with fixture-assisted time advancement. No committed artifact ties a worker crash or poison job to the kind rollout incident. The worker exercises therefore supplement the incident findings but should not be represented as a cluster-level incident response.

Source · **observed**: `docs/evidence/worker-20261113T092847Z-7e0a2f/summary.json`

6. Limitations and evidence gaps

The demonstrated Kubernetes environment is a single-laptop kind cluster, not a multi-node production cluster. Its capacity, failure domains, storage behavior, network path, and latency are not representative of a production region. The Prometheus p95 is a short local estimate from catalog GET traffic, not a load-tested service objective. Local kind runs on arm64 while CI builds linux/amd64; the published CI artifact and local image should not be assumed to be byte-identical across platforms.

Source · **observed / design intent**: `docs/environment-person-C.md`, `docs/evidence/prometheus-20261113T174220Z.txt`, `.github/workflows/ci.yml`

The operations evidence is intentionally bounded. The good rollout probe measures the local NodePort path during one deployment update; the broken-release probe measures the same service while a new revision is held unready. Neither exercise covers node loss, a full database outage, network partition, credential rotation, storage exhaustion, or a sustained traffic profile. Likewise, passing the public checker is a finite contract suite, not a formal proof of every possible request interleaving. Concurrency tests improve confidence in the specific races they exercise, while database constraints and transaction boundaries are the design controls intended to cover additional schedules.

Source · **observed / design intent**: `docs/evidence/rolling-update-20261113T180034Z/summary.md`, `docs/evidence/broken-release-20261113T180416Z/summary.md`, `tests/api/test_api.py`

Prometheus uses ephemeral storage in the local Kubernetes configuration, so its time series are not durable across pod replacement. The database Secret is created outside Git by the bootstrap script; this avoids committing credentials but leaves secret distribution and rotation as operator work. The local lab workflow uses broad cluster access; a production deployment would need narrowly scoped RBAC and a managed secret solution. Digest-pinned deployment references and stronger cluster access controls should be retained when moving beyond kind.

Source · **design intent**: `infra/k8s/base/prometheus/deployment.yaml`, `scripts/bootstrap-secrets.sh`, `docs/gitops.md`, `docs/threat-model.md`

Several rubric items remain explicitly unevidenced or only partially evidenced in the repository at report time:

Evidence gap	Status at report time
WP3 application-level red-to-green: a business/API test first fails, then passes after a fix	Not yet evidenced. PR #14 proves a dependency vulnerability gate, not a business-test bug fix. [`docs/evidence/red-green.md`]
Worker crash recovery and poison-job scenario in Compose or kind	Partially evidenced by four passing local process/PostgreSQL scenarios; no committed container- or Kubernetes-level worker kill/poison drill. [`docs/evidence/worker-20261113T092847Z-7e0a2f/summary.json`]
Branch protection settings	Configured by the repository admin; not re-readable through the API with the authors' write-level token, so the settings themselves are not attested in this repository. [`docs/ci-cd.md`]
Production availability, sustained load, multi-node recovery, and durable Prometheus history	Not tested; the available operations evidence is local kind/Compose. [`docs/evidence/README.md`]

These gaps distinguish implemented controls from demonstrated outcomes. A manifest can specify a policy, and a test can cover a code path, without proving behavior under a different runtime or a real failure. The dated requirements audit is a useful snapshot but predates several later evidence additions; current conclusions above use the current main-branch tree and current CI run rather than treating that older audit as the final state.

Source · `docs/requirements-audit.md`, commit `dc6fee8…`

7. Credits and tools

FulfillOps was built by a three-person team. Person A owned API behavior, schema and migrations, transactional order/idempotency behavior, and API tests. Person B owned worker processing, leases/retries, worker metrics, and worker reliability tests. Person C integrated containers and Compose, CI and image publishing, kind/Kustomize/Argo CD, Prometheus collection, release automation, and operational drills. Shared interfaces and runtime assumptions are captured in the integration contract.

Source · Project ownership: `AGENTS.md`, `docs/integration-contract.md`; Person A contribution record: `docs/individual-contribution-peushi.md`

The implementation uses Python 3.12, FastAPI, SQLAlchemy/psycopg, Alembic, PostgreSQL, Prometheus client and server, Docker/Compose, kind, Kustomize, Argo CD, GitHub Actions, Ruff, pytest, Trivy, Syft, and GHCR. Versions, image references, and workflow action SHAs are recorded in the environment files, lockfile, manifests, and workflow.

Source · `.github/workflows/ci.yml`, `requirements.lock`, `docs/environment.md`, `docs/environment-person-C.md`

AI assistance was used for parts of the infrastructure implementation, evidence collection and the drafting of this report. The repository's assistance log records each task, what was generated, and what a team member reviewed or changed; every author remains responsible for the claims about their own work.

Source · `docs/assistance-log.md`